TinyAya Tool-Calling Evaluation Report

Evaluating 3.35B-parameter multilingual models for structured tool calling across English, Swahili, and Luganda

Project	Tiny Facade
Models Evaluated	TinyAya family (5 variants: Base, Global, Earth, Fire, Water)
Languages	English, Swahili, Luganda
Test Cases	33 (30 tool calls + 3 refusals)

1. Objective

This report evaluates Cohere's TinyAya model family (3.35B parameters, 5 variants) for tool-calling capability across English, Swahili, and Luganda. The evaluation establishes a baseline understanding of how well these compact multilingual models handle structured tool calling without dedicated function-calling training.

2. Background

Tool calling is a core capability for on-device AI assistants. Rather than answering directly, the model must recognise when a user's request requires an external tool (weather lookup, calculator, web search, etc.) and output a structured JSON call with the correct tool name and arguments.

For a bilingual Luganda assistant, the challenge is cross-lingual: the user speaks Luganda, but the tool call must be in structured English JSON. For example:

User (Luganda)	`"Obudde bw'omu Kampala buli butya?"`
Expected Output	`{"name": "get_weather", "arguments": {"location": "Kampala"}}`

No existing benchmark covers Luganda tool calling, so a custom evaluation set was constructed.

3. Evaluation Design

3.1 Test Set

33 test cases across 3 languages and 10 tools:

Table 1: Test set composition

Category	Count	Examples
English tool calls	10	e.g. "What's the weather in Kampala?"
Swahili tool calls	10	e.g. "Hali ya hewa Dar es Salaam ikoje?"
Luganda tool calls	10	e.g. "Obudde bw'omu Kampala buli butya?"
Refusals	3	e.g. "Tell me a joke about programming"

The 10 tools include: `get_weather`, `search_web`, `calculate`, `translate_text`, `set_reminder`, `get_news`, `convert_currency`, `send_message`, `create_event`, and `get_directions`.

3.2 Prompt Styles

Each model was tested under two prompting strategies:

- **Explicit:** The system prompt specifies the expected JSON format and lists available tools with their parameters.

- **Implicit:** Only the tool schemas are provided with no format instructions.

3.3 Scoring Metrics

Table 2: Scoring metrics

Metric	Description
JSON Valid	Can the model's response be parsed as valid JSON?
Name Correct	Did the model select the right tool?
Arg F1	Overlap between expected and actual arguments (precision/recall)
Refusal Accuracy	For non-tool queries, did the model correctly avoid calling a tool?

3.4 Multilingual Approach

Swahili and Luganda test queries are natural translations of the English queries, written as a voice assistant user would speak them. The expected output is always English JSON. The model must bridge from the user's language to structured tool calls. Swahili was included because TinyAya was explicitly trained on it, while Luganda tests zero-shot transfer via Bantu language similarity.

4. Models Tested

Table 3: TinyAya model variants (all 3.35B params, Cohere2 architecture)

Variant	Description
tiny-aya-base	Pretrained base model, 70+ languages
tiny-aya-global	Instruction-tuned for balanced multilingual performance
tiny-aya-earth	Regional variant optimised for African and West Asian languages
tiny-aya-fire	Regional variant optimised for South Asian languages
tiny-aya-water	Regional variant optimised for Asia-Pacific and European languages

Each variant was tested at both full precision (fp16/bf16) and 4-bit quantisation, giving 20 total configurations.

5. Results

5.1 Implicit Prompting

0% across all models. No TinyAya variant can infer tool-calling format from schemas alone. This confirms none have been trained for function calling.

5.2 Explicit Prompting: Per-Language Breakdown

The table below shows JSON validity, tool name accuracy, and argument F1 for each language. All models are ranked by overall JSON validity across the 30 tool-call queries (10 per language).

Table 4: Full per-language results (explicit prompting, 10 queries per language)

Model	Quant	EN JSON	EN Name	EN F1	SW JSON	SW Name	SW F1	LG JSON	LG Name	LG F1
global	**4bit**	**70%**	**70%**	**0.63**	**90%**	**70%**	**0.60**	**70%**	**40%**	**0.37**
water	4bit	40%	40%	0.27	80%	60%	0.50	60%	0%	0.00
base	fp16	30%	10%	0.21	70%	30%	0.43	30%	20%	0.17
global	fp16	30%	30%	0.27	70%	60%	0.53	20%	20%	0.20
water	fp16	20%	20%	0.17	70%	50%	0.43	20%	10%	0.07
base	4bit	40%	20%	0.27	50%	30%	0.23	10%	10%	0.10
fire	fp16	40%	40%	0.33	50%	50%	0.43	10%	10%	0.10
earth	4bit	10%	10%	0.07	40%	20%	0.17	20%	10%	0.10
earth	fp16	20%	20%	0.11	40%	40%	0.37	10%	10%	0.10
fire	4bit	30%	30%	0.30	30%	30%	0.30	10%	10%	0.10

*JSON = valid JSON output, Name = correct tool selected, F1 = argument F1 score. CI is approximately +/-17% at n=10 per language. Quant column: **amber = 4bit**, **blue = fp16**.*

5.3 Overall Summary

Aggregated across all 30 tool-call queries (3 languages combined):

Table 5: Overall results summary, ranked by JSON validity

Rank	Model	Quant	JSON %	Name %	Arg F1
1	**global**	**4bit**	**77%**	**60%**	**0.53**
2	water	4bit	60%	33%	0.26
3	base	fp16	43%	20%	0.27
4	global	fp16	40%	37%	0.33
5	water	fp16	37%	27%	0.22
6	base	4bit	33%	20%	0.20
7	fire	fp16	33%	33%	0.29
8	earth	4bit	23%	13%	0.11
9	earth	fp16	23%	23%	0.19
10	fire	4bit	23%	23%	0.23

5.4 Refusal Accuracy

For the 3 non-tool queries (e.g. "Tell me a joke about programming"), the model should avoid calling any tool. Refusal accuracy measures how often each configuration correctly abstained.

Table 6: Refusal accuracy (3 non-tool queries)

Model	Quant	Refusal Accuracy
global	fp16	100%
water	fp16	100%
earth	4bit	100%
water	4bit	67%
base	fp16	67%
fire	fp16	67%
base	4bit	67%
fire	4bit	67%
global	4bit	33%
earth	fp16	33%

Notably, the best tool-calling configuration (global 4bit, 77% JSON validity) has among the worst refusal accuracy (33%), suggesting an inverse relationship between tool-calling eagerness and appropriate restraint.

6. Key Findings

1. **Swahili consistently outperforms English and Luganda** across all models (e.g. global 4bit: SW 90% vs EN 70% vs LG 70%). This reflects TinyAya's training distribution, where Swahili is one of their target African languages.

2. **4-bit quantisation dramatically improves some models.** The global variant jumps from 40% to 77% with quantisation. This is counterintuitive and may reflect regularisation effects or noise from the small test set (n=30 tool cases, +/-17% CI).

3. **None of these models have native tool-calling ability.** Implicit prompting (providing only tool schemas) yields 0% across all 5 variants. They require explicit format instructions in the system prompt.

4. **The dominant failure mode is repetition.** Models often produce the correct JSON on their first attempt but then repeat it in a loop, breaking the parser. This is a known issue with small autoregressive models.

7. Limitations

- **Small test set:** 30 tool-call cases gives +/-17% confidence intervals. Results are directional, not publication-grade.
- **Custom eval only:** Standardised benchmarks (BFCL, MASSIVE-Agents) have not yet been run on these models.
- **Single evaluation:** No repeated runs to measure variance.

8. Next Steps

- **GRPO training** (in progress): Reinforcement learning on 19,938 bilingual QA examples with real web search, teaching the model when and how to call tools.
- **MASSIVE-Agents evaluation:** 2,792 gold-standard examples per language (EN, SW, LG) for statistically meaningful comparison.
- **Post-training comparison:** Re-evaluate all TinyAya variants on the full benchmark after additional fine-tuning experiments.